Alert Policies for Industrial Visual Inspection at the Edge: An Integrated Evaluation of Temporal Gating, Sensor Fusion and Durable Event Delivery

Utkarsh Sengar
Department of Computer Science and Engineering
Indian Institute of Technology Dharwad
Dharwad, Karnataka 580011, India
cs23bt026@iitdh.ac.in

***Abstract*—Visual anomaly detectors produce a score per frame, but an inspection line needs alerts an operator can act on. We present and evaluate an edge inspection runtime that adapts established alarm-management primitives (smoothing, k-of-N persistence, state-based suppression and an incident latch) to PatchCore scores, combines visual and simulated physical evidence for severity routing, and stores every event in a SQLite spool until the MQTT broker acknowledges it. We do not propose a new algorithm; the contribution is an integrated, reproducible evaluation. On 6 workloads driven by real PatchCore scores on 7 MVTec AD categories (1,386 runs), we compare 8 variants of the policy with 3 external alarm baselines and report category-level confidence intervals. With frame scores drawn independently, the full policy raised no false alert on good parts (single-frame thresholding: 4,616 per hour), one alert per sustained defect, and 4.0 false line-stop escalations per hour, fewer than any external baseline. A textbook delay-timer suppressed short glare far better (0.03 vs. 0.66 alerts per burst) but caught only 0.07 of 3-frame defects (full policy: 0.70). When consecutive scores were correlated, every persistence-based policy lost its zero false-alert result (full policy: 188.6 per hour). On 12 held-out VisA categories, evaluated with the policy frozen, the advantages from machine-state handling and the delay-timer's glare advantage replicated, the full policy's line-stop advantage over decision fusion did not, and recall of sustained defects within 0.5 s fell to 0.77 (decision fusion: 0.52). Against a real broker the spool lost no event under outages of up to 120 s, a broker restart or a publisher crash (at-least-once delivery), and 100 crashes of the inspection process lost no decision. Subscriber crashes did lose events: all 300 sent during a 10 s outage with a clean session, and 807 around a 60 s outage with a persistent session under the broker's default queue limit, which the spool cannot prevent. On a laptop GPU, moving database writes to a writer thread reduced 30 FPS frame-budget misses from 2.46% to 1.00% of cycles, but did not remove them.**

***Index Terms*—Edge computing, visual anomaly detection, alarm management, sensor fusion, MQTT, store-and-forward.**

I. Introduction

Unsupervised detectors such as PatchCore [1] and PaDiM [2] rank images well on MVTec AD [3], [4], and edge accelerators can run them at line rate [5]. Ranking quality is not the same as a usable alarm stream. A camera at 30 frames per second sees the same defect in dozens of consecutive frames; reflections produce short score spikes; and scores are meaningless while the lens is blurred or the machine is being serviced. Process-industry alarm management has studied the resulting overload for decades: ISA-18.2 [6] and EEMUA 191 [7] recommend on-delay timers, deadbands and state-based alarming, and the trade-off between false alarms and detection delay of delay-timers, generalized k-of-N timers and deadbands has been analyzed in detail [8]–[12]. Networking adds a second problem: an alert lost during a broker outage is worse than a late one.

This paper does not claim a new filter or fusion rule. It asks how these established primitives behave when they are wrapped around a modern visual anomaly detector and an edge delivery path, and measures alert burden, detection delay, short-defect recall, latency and outage behavior in one prototype. Our contributions are:

1) an edge inspection runtime that adapts established alarm-management primitives to visual anomaly scores, routes severity using visual and physical evidence, and handles degraded inputs explicitly;
2) an evaluation on real PatchCore score streams against 3 external alarm baselines (delay-timer, EMA with hysteresis, decision-level fusion), with per-category results, category-level confidence intervals and permutation tests, a rerun with temporally correlated scores, and a held-out rerun on the 12 VisA categories with the policy frozen;
3) failure tests of the delivery path against a real MQTT broker (link loss, broker restart, publisher, subscriber and inspection-process crashes, spool overflow), and per-stage latency with a real forward pass, with and without a persistence writer thread.

The results are mixed, and we report them as such: the integrated policy gave the fewest false line-stop escalations and handled machine states and degraded inputs, but a simple delay-timer suppressed short glare better, all persistence-based policies were sensitive to the independence assumption of our score sampling, and on the harder VisA images every policy except single-frame thresholding missed a substantial share of sustained defects within 0.5 s.

II. Related Work

Alarm management. Delay-timers, deadbands and their generalized k-of-N forms reduce nuisance alarms at the cost of detection delay; expected delays and false/missed alarm

rates have closed forms for simple signal models [8]–[10]. Later work designs delay timers from the distribution of alarm durations [13], uses multiple set points [14], combines deadbands and delay timers in serial alarm systems [15] or selects them jointly from risk indicators [16], and analyzes on/off-delay timers for stochastic anomaly detectors [17]; surveys document alarm floods and state-based alarming [11], [12]. These works target process variables or generic detector outputs, not image-level anomaly scores from a camera. *Event-level evaluation.* Evaluating anomaly detectors by events under an alarm budget, rather than by points, has been advocated for industrial control systems [18] and industrial multivariate time series [19], and alarm onsets have been used to trigger fault diagnosis in collaborative robots [20]. We adopt this perspective for visual inspection. *Visual anomaly detection.* PatchCore [1] and PaDiM [2] are standard on MVTec AD [3]; VisA [21] and Real-IAD [22] are closer to production images. Surveys and benchmarks of industrial anomaly detection [23], [24], work on efficient detection on resource-constrained edge devices [25] and multimodal edge defect detection with Bayesian fusion [26] report detection accuracy and cost, not the alert stream an operator receives. *Edge messaging.* MQTT [27] with QoS 1 provides at-least-once delivery between client and broker; edge brokers have been compared for throughput, latency and resilience [28], [29], MQTT-based edge architectures are used for real-time monitoring [30], and edge-based store-and-forward has been proposed for resilient IIoT communication [31]. *Positioning.* None of the alarm primitives, the fusion rule or the spool is new. The contribution is empirical: the primitives are adapted to image-level anomaly scores with explicit handling of degraded inputs and machine states, and the whole chain from score to delivered alert is evaluated in one prototype, against external alarm baselines, on a second dataset with everything frozen, and under failures of the publisher, the broker, the subscriber and the inspection process. To our knowledge these aspects are rarely evaluated together; this combination, not any single component, is the subject of this paper, and we make no claim that the integrated policy is generally superior.

III. System

A. Scoring

PatchCore (ResNet-18 layer2/3 features, 10% greedy k-center coreset [32], 224×224 input) returns the maximum of a smoothed nearest-neighbor distance map. We normalize each category with $v = \min(1, 0.5\,d/d_{99})$, where d_{99} is the 99th percentile of distances on held-out *training* images. Before scoring, an optical check computes the variance of the Laplacian [33] and the mean brightness; frames below a per-category blur threshold (half the 5th percentile on held-out training images) or outside fixed brightness limits skip the detector. Vibration, temperature and current are standardized against their running-state baselines and fused into $s = 1 - \exp(-\sum_c w_c \max(0, (z_c - 3)/3))$ with weights $0.45/0.25/0.30$.

B. Decision policy

Scores are smoothed with $\bar{v}_t = \alpha_v v_t + (1 - \alpha_v)\bar{v}_{t-1}$ and $\bar{s}_t = \alpha_s s_t + (1 - \alpha_s)\bar{s}_{t-1}$ ($\alpha_v = 0.35$, $\alpha_s = 0.25$), each initialized with its first valid sample. A frame that fails the optical check does not update $\bar{v}$, and a reading with a missing channel does not update $\bar{s}$. Visual evidence is *high* when at least $k = 4$ of the last $N = 10$ smoothed scores reach $\tau_{\text{high}} = 0.8$ and *medium* for the same test at $\tau_{\text{med}} = 0.5$; sensor evidence is confirmed by the same test at $\tau_{\text{phys}} = 0.7$. The candidate decision is chosen in order: (1) a machine in FAULT state yields HIGH; (2) if the camera is degraded, confirmed sensor evidence yields REVIEW, an optical failure in k of the last N frames yields a REVIEW for camera maintenance, and shorter failures are held silently; (3) high visual evidence confirmed by the sensors yields HIGH; (4) high visual evidence without confirmed sensor evidence yields REVIEW if a channel is missing, or if $|\bar{v}_t - \bar{s}_t| \geq \tau_{\text{div}} = 0.45$ or $\bar{v}_t < \tau_{\text{high}}$ (cross-modal discrepancy), and HIGH otherwise; (5) confirmed sensor evidence alone, or medium visual evidence, yields REVIEW. In IDLE or MAINTENANCE the candidate is lowered by one level. The condition $\bar{v}_t < \tau_{\text{high}}$ in rule (4) was added after our first full ablation: without it, the k-of-N window still reported high evidence for several frames after a glare burst had ended while $|\bar{v}_t - \bar{s}_t|$ had already fallen below τ_{div}, so decaying glare escalated to HIGH on nominal sensors. All numeric parameters were fixed before the experiments; all results below use the revised rule.

C. Incident latch

Each non-normal decision either opens an incident, which emits one alert, or joins the open incident silently; a severity increase inside an incident emits one more alert. The incident closes after $T_{\text{cool}} = 15$ consecutive normal frames (0.5 s). Only alerts enter the operator queue; every decision is still written to the audit log.

D. Durable delivery

Every QoS 1 event is inserted into a SQLite spool in WAL mode with `synchronous=NORMAL` [34], keyed by a unique event identifier. One drain thread publishes spooled events in insertion order over MQTT 3.1.1 [27] and deletes a row only in the `on_publish` callback, i.e. after the broker's PUBACK. This gives at-least-once delivery to the broker: a crash between PUBACK and deletion re-sends the event, and the subscriber and audit log discard repeated identifiers. The subscriber that writes the audit log connects with a fixed client identifier and a persistent session, so the broker queues QoS 1 events while it is down and re-delivers them when it reconnects; it acknowledges an event only after the handler that stores it has returned (a failed insert is logged and still acknowledged). It does not give exactly-once processing downstream, and `synchronous=NORMAL` may lose the last transactions on power loss. At capacity the spool evicts its oldest rows and counts them. Spool and audit inserts run either on the inspection

thread (*synchronous*) or on one *writer thread* fed by an in-memory queue; with the writer thread, decisions still in the queue are lost if the process crashes.

IV. Evaluation Method

A. Score banks

For each of the 7 categories we fitted PatchCore on a random 80% of the good training images and used the remaining 20% to calibrate d_{99} and the blur threshold. We scored every test image, and every good test image three more times with a synthetic specular highlight (a blurred saturated ellipse). Image AUROC ranged from 0.925 to 1.000. On 5 of the 7 categories the median normalized defect score lay below τ_{high}, and the median glare score was at least 0.57 in every category. The pools are small: 20–58 good test images per category.

B. Workloads

Each workload lasts 9,000 frames (5 min at 30 Hz) and places randomized events without overlap (sensor drift and dropout may overlap other events): good parts only; 40 glare bursts of 1–3 frames plus short blur; eight defects of 2–8 s; defects with a mechanical fault, mechanical faults alone and 10 glare events of up to 2 s; defects around blur, darkness, current dropout and a temperature drift ramp; and defects between idle and maintenance windows in which glare and mechanical load occur. Per frame, the visual score is drawn from the pool matching the timeline, the optical check runs on a held-out training frame that is blurred or darkened when the timeline says so, and the sensor simulator follows the timeline's faults. Defects and mechanical faults while running are ground truth. In the main runs every frame's score is drawn independently. A second set of 1,386 runs replays the same timelines with temporally correlated scores: a latent AR(1) process with coefficient $\rho = 0.9$ per frame is mapped through a Gaussian copula to quantiles of the current pool, so marginals are unchanged but consecutive frames are similar.

C. Policies

`BASELINE` thresholds every raw score ($v \geq \tau_{\text{high}}$: HIGH, $v \geq \tau_{\text{med}}$: REVIEW); `EMA_ONLY` applies the thresholds to $\bar{v}$; `EMA_KOFN` adds the k-of-N test. These alert on every non-normal frame. `FULL_POLICY` is the policy of Section III; `NO_COOLDOWN`, `NO_FUSION`, `NO_DIVERGENCE` and `NO_STATE_GATING` each remove one component from it. The external baselines are textbook alarms using the same thresholds, with no parameter tuned on our results: `DELAY_TIMER` applies an on-delay of k consecutive raw samples and an off-delay of T_{cool} samples to each level [6], [9]; `EMA_HYSTERESIS` applies a deadband of 0.1 to $\bar{v}$ [10]; and `DECISION_FUSION` combines a visual delay-timer alarm (at τ_{med}) and a sensor delay-timer alarm (at τ_{phys}) into HIGH if both are active and REVIEW if one is. They alert when their alarm severity rises, the same alert definition as the latch, and they use neither the optical check nor machine states.

D. Metrics and statistics

An *alert* is a decision that enters the operator queue. Alerts within an episode or up to 15 frames after it belong to that episode; other alerts are false (*critical* if at HIGH). Recall is the fraction of episodes with any non-normal decision within 15 frames of onset, and delay is measured to that decision. False alerts are normalized by the time outside episodes, and glare responses by the number of bursts. An experimental unit is a (category, replicate) pair with three replicates per category; all policies see identical inputs per unit (21 paired units per cell). Replicates of one category draw from the same small pools and are not independent, so every interval in the tables is a 95% percentile bootstrap that resamples *categories* ($B = 2{,}000$) [35]. For comparisons we report the mean paired difference per unit with this interval, the number of categories in which the full policy was better and worse, and an exact two-sided sign-flip permutation test on the category-mean differences (all 2^n sign assignments for n categories). With seven categories the smallest attainable p is 0.016, so these tests can only detect differences that are consistent in direction across categories.

V. Results

A. Policy ablation

Table I summarizes all policies. With good parts only, single-frame thresholding produced 4,616 false alerts per hour (category interval 1,557–8,374), because normalized scores of good test images cross 0.5; this mean is driven by a few categories (2 categories had none, metal_nut had 14,780). Smoothing alone left 40.6 per hour and the full policy none. Persistence without a latch *increased* alerts per glare burst (5.66 vs. 2.63), because the smoothed score of a 3-frame burst stays above τ_{med} long enough to pass the non-consecutive k-of-N test; the latch brought them to 0.66. The latch also turned a sustained defect from 137.64 alerts into 1.00, at a mean delay of 3.7 frames (122 ms), and every sustained defect was flagged within 15 frames (recall 1.00; delay-timer and decision fusion 0.96). Without sensor fusion, multi-modal recall fell from 1.00 to 0.62, since mechanical faults without a visible defect were never seen. Divergence triage lowered false HIGH escalations from 117.0 to 4.0 per hour, and state gating lowered false alerts around maintenance from 121.8 to 1.3 per hour. In the degraded-input workload 147 of the 314 alerts of the full policy were camera-maintenance requests for sustained blur or darkness, which we count as false, and 42 were caused by the temperature drift, which the policy cannot tell apart from real heating; without the maintenance requests its rate would be 26.9 per hour.

B. Comparison with external baselines

Table II compares the full policy with the baselines. Against `DECISION_FUSION`, the closest simple alternative, the full policy had slightly fewer false line-stops (difference 1.4 per hour, interval 0.0 to 3.4), the same multi-modal recall, about the same number of alerts per sustained defect (0.98 vs. 1.00), and far fewer false alerts around machine states (124.4 per hour, better in 7 of 7 categories), because the baselines have no

TABLE I

MEANS OVER 21 PAIRED UNITS (INDEPENDENT FRAME SCORES). FA/H: FALSE ALERTS PER HOUR OUTSIDE EPISODES. ALERTS/BURST: ALERTS PER 1–3 FRAME GLARE BURST. ALERTS/EP.: ALERTS PER SUSTAINED-DEFECT EPISODE (1 IS IDEAL). RECALL: SHARE OF EPISODES WITH A NON-NORMAL DECISION WITHIN 15 FRAMES OF ONSET (SUSTAINED DEFECTS; MULTI-MODAL FAULTS). CRIT. FA/H: FALSE ALERTS AT HIGH. ROWS BELOW THE LINE ARE EXTERNAL BASELINES. CATEGORY-LEVEL INTERVALS ARE IN TABLES II AND III.

	Nominal	Glare	Sustained defects			Multi-modal faults		Degraded	States
Policy	FA/h	alerts/burst	alerts/ep.	delay (fr.)	recall	recall	crit. FA/h	FA/h	FA/h
BASELINE	4,616	2.63	137.64	0.1	1.00	0.78	3,342	4,253	7,299
EMA_ONLY	40.6	4.27	149.21	0.6	1.00	0.62	3,229	37.6	3,179
EMA_KOFN	0.0	5.66	153.95	3.7	1.00	0.62	3,649	0.0	3,389
NO_COOLDOWN	0.0	5.66	153.95	3.7	1.00	1.00	108.1	18,526	5.7
NO_FUSION	0.0	0.66	1.38	3.7	1.00	0.62	117.7	107.5	66.2
NO_DIVERGENCE	0.0	0.66	1.38	3.7	1.00	1.00	117.0	120.9	66.2
NO_STATE_GATING	0.0	0.66	1.00	3.7	1.00	1.00	4.0	120.9	121.8
FULL_POLICY	0.0	0.66	1.00	3.7	1.00	1.00	4.0	120.9	1.3
DELAY_TIMER	5.1	0.03	1.23	3.9	0.96	0.61	104.2	5.1	107.0
EMA_HYSTERESIS	3.4	0.59	1.58	0.4	1.00	0.84	125.1	11.5	105.5
DECISION_FUSION	5.1	0.03	0.98	3.9	0.96	1.00	5.4	30.7	125.6

TABLE II

PAIRED DIFFERENCES, BASELINE MINUS FULL_POLICY, PER UNIT: MEAN [95% CATEGORY-CLUSTER BOOTSTRAP INTERVAL] (NUMBER OF CATEGORIES IN WHICH FULL_POLICY WAS BETTER : WORSE, THE REMAINDER TIED; EXACT SIGN-FLIP PERMUTATION p OVER THE 7 CATEGORIES, SMALLEST ATTAINABLE 0.016). POSITIVE VALUES FAVOR FULL_POLICY EXCEPT FOR RECALL, WHERE NEGATIVE VALUES DO.

Metric	BASELINE	DELAY_TIMER	EMA_HYSTERESIS	DECISION_FUSION
nominal FA/h	4,616 [1,557, 8,374] (5:0; $p = 0.062$)	5.1 [0.0, 15.4] (1:0; $p = 1.000$)	3.4 [0.0, 6.9] (2:0; $p = 0.500$)	5.1 [0.0, 15.4] (1:0; $p = 1.000$)
glare alerts/burst	1.97 [1.59, 2.44] (7:0; $p = 0.016$)	-0.64 [-0.79, -0.42] (0:7; $p = 0.016$)	-0.07 [-0.32, 0.19] (3:4; $p = 0.656$)	-0.64 [-0.79, -0.42] (0:7; $p = 0.016$)
alerts/ep.	136.64 [124.15, 148.53] (7:0; $p = 0.016$)	0.23 [0.02, 0.47] (4:1; $p = 0.188$)	0.58 [-0.41, 1.93] (4:3; $p = 0.562$)	-0.02 [-0.05, 0.00] (0:1; $p = 1.000$)
delay (fr.)	-3.6 [-4.0, -3.2] (0:7; $p = 0.016$)	0.3 [-0.4, 1.0] (3:4; $p = 0.562$)	-3.3 [-3.5, -3.1] (0:7; $p = 0.016$)	0.3 [-0.4, 1.0] (3:4; $p = 0.562$)
defect recall	0.00 [0.00, 0.00] (0:0; $p = 1.000$)	-0.04 [-0.12, 0.00] (1:0; $p = 1.000$)	0.00 [0.00, 0.00] (0:0; $p = 1.000$)	-0.04 [-0.12, 0.00] (1:0; $p = 1.000$)
fault recall	-0.22 [-0.32, -0.12] (6:0; $p = 0.031$)	-0.39 [-0.46, -0.34] (7:0; $p = 0.016$)	-0.16 [-0.28, -0.05] (5:0; $p = 0.062$)	0.00 [0.00, 0.00] (0:0; $p = 1.000$)
crit. FA/h	3,338 [1,947, 4,585] (6:0; $p = 0.031$)	100.1 [61.3, 130.3] (6:0; $p = 0.031$)	121.1 [77.9, 150.2] (6:0; $p = 0.031$)	1.4 [0.0, 3.4] (2:0; $p = 0.500$)
degraded FA/h	4,132 [1,288, 7,592] (5:2; $p = 0.062$)	-115.8 [-122.2, -108.1] (0:7; $p = 0.016$)	-109.4 [-122.0, -89.1] (0:7; $p = 0.016$)	-90.2 [-97.6, -81.6] (0:7; $p = 0.016$)
states FA/h	7,298 [4,322, 10,961] (7:0; $p = 0.016$)	105.8 [87.3, 126.4] (7:0; $p = 0.016$)	104.2 [63.1, 140.1] (6:0; $p = 0.031$)	124.4 [117.5, 134.4] (7:0; $p = 0.016$)

notion of machine state. The baselines, however, were better on two metrics. A delay-timer that requires k *consecutive* raw samples above threshold rejected almost every 1–3 frame glare burst (0.03 vs. 0.66 alerts per burst; FULL_POLICY worse in 7 of 7 categories), whereas smoothing spreads a burst over enough frames to pass the non-consecutive k-of-N test. In the degraded-input workload all three baselines raised fewer false alerts (5.1, 11.5 and 30.7 vs. 120.9 per hour). Most of the difference comes from the camera-maintenance requests, which the baselines never raise because a blurred frame simply scores zero for them; without them the full policy's rate was 26.9 per hour, still above the two vision-only baselines, which also cannot see the temperature drift. EMA_HYSTERESIS detected sustained defects sooner than the full policy (0.4 vs. 3.7 frames) but missed mechanical faults (recall 0.84) and raised 125.1 false line-stops per hour.

C. Per-category results

Table III lists the full policy per category. False alerts on good parts, alerts per sustained defect and multi-modal recall were the same in every category. The mean delay ranged from 3.0 to 5.0 frames, critical false alerts were zero in some categories and not in others, and glare alerts per burst were lowest for cable, the category whose median glare score was also the lowest (0.57). The degraded-input rate was similar in all categories because it is dominated by camera-maintenance and drift alerts, which do not depend on the visual scores. Fig. 1 shows the raw category-level differences behind Table II; per-category values for every metric and comparator are in the artifact.

D. Temporally correlated scores

Our main runs draw each frame's score independently, which real video does not do. Table IV repeats the ablation with $\rho = 0.9$. The zero false-alert result on good parts does not survive: with correlated scores, runs of good frames above τ_{med} pass every persistence test, and the full policy raised 188.6 false alerts per hour (DELAY_TIMER: 300.0, EMA_HYSTERESIS: 105.1). Alerts per sustained defect rose slightly (1.07) and sustained-defect recall fell to 0.95, while the comparisons on glare and line-stops kept their direction: the delay-timer still had the fewest alerts per glare burst (0.06 vs. 0.63), and the full policy the fewest critical false alerts (3.4 vs. 21.4 for decision fusion). False-alert rates on good parts in Table I should therefore be read as optimistic.

E. Held-out evaluation on VisA

To test whether the conclusions, and the revised rule (4), survive data that played no part in the design, we froze all code and parameters and then built score banks for the 12 VisA

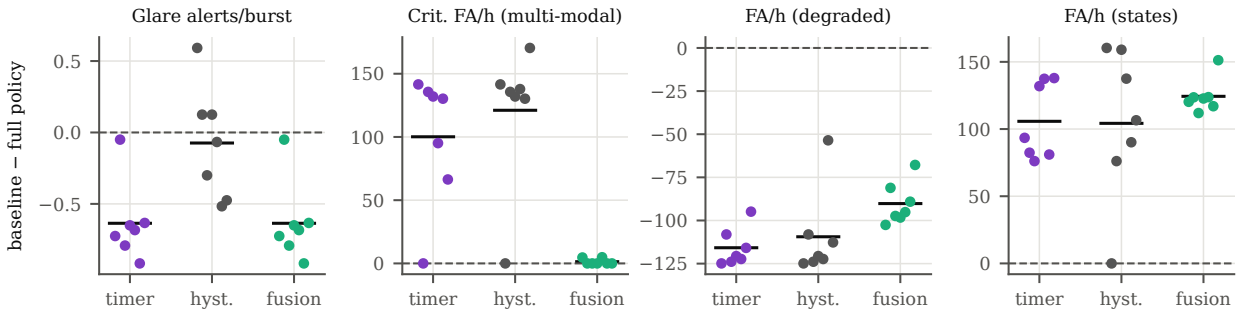


Fig. 1. Raw category-level paired differences, baseline minus FULL_POLICY (one dot per MVTec category, bar: mean; positive values favor FULL_POLICY). Timer: DELAY_TIMER; hyst.: EMA_HYSTERESIS; fusion: DECISION_FUSION.

TABLE III
FULL_POLICY PER CATEGORY (MEANS OVER THREE REPLICATES, INDEPENDENT FRAME SCORES) AND 95% CATEGORY-CLUSTER BOOTSTRAP INTERVAL OF THE OVERALL MEAN.

	Nominal	Glare	Sustained defects			Multi-modal faults		Degraded	States
Category	FA/h	alerts/burst	alerts/ep.	delay (fr.)	recall	recall	crit. FA/h	FA/h	FA/h
bottle	0.0	0.72	1.00	3.1	1.00	1.00	0.0	124.9	0.0
cable	0.0	0.05	1.00	4.2	1.00	1.00	0.0	108.1	0.0
carpet	0.0	0.85	1.00	3.5	1.00	1.00	4.6	123.9	8.9
grid	0.0	0.67	1.00	5.0	1.00	1.00	9.5	120.6	0.0
hazelnut	0.0	0.68	1.00	3.7	1.00	1.00	4.6	122.3	0.0
leather	0.0	0.97	1.00	3.0	1.00	1.00	9.5	115.9	0.0
metal_nut	0.0	0.69	1.00	3.1	1.00	1.00	0.0	130.9	0.0
95% CI	[0.0, 0.0]	[0.44, 0.83]	[1.00, 1.00]	[3.2, 4.2]	[1.00, 1.00]	[1.00, 1.00]	[1.3, 6.8]	[115.9, 125.7]	[0.0, 3.8]

TABLE IV
THE SAME WORKLOADS WITH TEMPORALLY CORRELATED VISUAL SCORES (AR(1) GAUSSIAN COPULA, $\rho = 0.9$ PER FRAME; MEANS OVER 21 UNITS). COLUMNS AS IN TABLE I.

	Nominal	Glare	Sustained defects			Multi-modal faults		Degraded	States
Policy	FA/h	alerts/burst	alerts/ep.	delay (fr.)	recall	recall	crit. FA/h	FA/h	FA/h
BASELINE	4,615	2.60	136.26	0.2	0.98	0.67	3,335	4,221	7,191
EMA_KOFN	2,438	6.05	146.19	3.7	0.95	0.61	3,544	2,255	5,676
FULL_POLICY	188.6	0.63	1.07	3.7	0.95	1.00	3.4	276.5	103.1
DELAY_TIMER	300.0	0.06	1.58	3.4	0.95	0.64	111.6	277.2	370.0
EMA_HYSTERESIS	105.1	0.66	1.79	0.7	0.98	0.79	116.3	122.3	209.1
DECISION_FUSION	300.0	0.06	1.04	3.4	0.95	1.00	21.4	302.8	390.7

categories [21] with the identical procedure on the official one-class split, and reran every workload (2,376 runs over 36 units); nothing was changed after looking at these results. Image AUROC ranged from 0.692 to 0.989, and the median normalized defect score lay below τ_{high} in 11 and below τ_{med} in 5 of the 12 categories. Tables V and VI report the results.

Three findings replicated. State gating kept false alerts around machine states at 2.9 per hour against 107.1–122.6 for the baselines (better in 12 of 12 categories, $p < 0.001$); divergence triage lowered false line-stops from 104.4 to 5.5 per hour; and the delay-timer again suppressed glare far better (0.01 vs. 0.55 alerts per burst, FULL_POLICY worse in 12 of 12 categories). Three did not. The full policy's line-stop advantage over decision fusion disappeared (5.5 vs. 5.9 per hour, $p = 1.000$); it raised 1.7 false alerts per hour on good parts where the delay-timer and decision fusion raised none; and with weak defect scores, incidents closed and reopened inside a defect (1.34 alerts per episode, 0.42 re-alerts). Most importantly, sustained-defect recall within 15 frames fell to 0.77 for the full policy, 0.52 for the delay-timer and decision fusion and 0.91 for EMA_HYSTERESIS, against 0.99 for single-frame thresholding: thresholds calibrated so that good parts reach 0.5 at their 99th percentile leave many VisA defects near or below τ_{med}, and persistence then delays or suppresses them. With correlated scores ($\rho = 0.9$) the full policy raised 111.0 false alerts per hour on good parts (decision fusion 125.7), kept the fewest false line-stops (3.5 vs. 16.5), and the full policy, the delay-timer and decision fusion all reached a defect recall of 0.64.

TABLE V

HELD-OUT EVALUATION ON THE 12 VISA CATEGORIES WITH THE POLICY FROZEN (INDEPENDENT FRAME SCORES; MEANS OVER 36 UNITS). COLUMNS AS IN TABLE I.

	Nominal	Glare	Sustained defects			Multi-modal faults		Degraded	States
Policy	FA/h	alerts/burst	alerts/ep.	delay (fr.)	recall	recall	crit. FA/h	FA/h	FA/h
BASELINE	2,029	2.28	82.10	1.3	0.99	0.69	3,071	1,878	4,832
EMA_ONLY	192.0	3.63	100.83	2.7	0.91	0.58	2,986	173.4	3,345
EMA_KOFN	10.3	4.45	111.92	5.7	0.77	0.49	3,436	8.6	3,415
NO_COOLDOWN	10.3	4.45	111.92	5.7	0.77	1.00	155.2	18,889	25.3
NO_FUSION	1.7	0.55	1.46	5.7	0.77	0.49	105.2	108.5	61.6
NO_DIVERGENCE	1.7	0.55	1.46	5.7	0.77	1.00	104.4	123.5	62.3
NO_STATE_GATING	1.7	0.55	1.34	5.7	0.77	1.00	5.5	123.5	123.4
FULL_POLICY	1.7	0.55	1.34	5.7	0.77	1.00	5.5	123.5	2.9
DELAY_TIMER	0.0	0.01	0.92	6.1	0.52	0.32	98.9	0.0	107.1
EMA_HYSTERESIS	3.3	0.74	1.74	2.0	0.91	0.72	127.6	14.7	122.6
DECISION_FUSION	0.0	0.01	0.78	6.1	0.52	1.00	5.9	25.8	121.2

TABLE VI

VISA: PAIRED DIFFERENCES, BASELINE MINUS FULL_POLICY, WITH 95% CATEGORY-CLUSTER INTERVALS, (CATEGORIES BETTER : WORSE) AND EXACT SIGN-FLIP PERMUTATION p OVER 12 CATEGORIES. SIGN CONVENTION AS IN TABLE II.

Metric	BASELINE	DELAY_TIMER	EMA_HYSTERESIS	DECISION_FUSION
nominal FA/h	2,027 [1,076, 3,092] (9:0; $p = 0.004$)	-1.7 [-4.7, 0.0] (0:2; $p = 0.500$)	1.7 [0.0, 5.0] (1:0; $p = 1.000$)	-1.7 [-4.7, 0.0] (0:2; $p = 0.500$)
glare alerts/burst	1.73 [1.57, 1.89] (12:0; $p < 0.001$)	-0.54 [-0.67, -0.38] (0:12; $p < 0.001$)	0.19 [-0.10, 0.45] (8:4; $p = 0.213$)	-0.54 [-0.67, -0.38] (0:12; $p < 0.001$)
alerts/ep.	80.76 [57.56, 103.69] (12:0; $p < 0.001$)	-0.42 [-1.06, 0.15] (3:6; $p = 0.219$)	0.40 [-0.89, 1.87] (5:7; $p = 0.591$)	-0.56 [-1.12, -0.06] (0:6; $p = 0.031$)
delay (fr.)	-4.8 [-5.8, -4.1] (0:11; $p < 0.001$)	0.9 [0.2, 1.6] (8:3; $p = 0.172$)	-4.0 [-5.2, -3.2] (0:11; $p < 0.001$)	0.9 [0.2, 1.6] (8:3; $p = 0.172$)
defect recall	0.22 [0.06, 0.41] (0:5; $p = 0.062$)	-0.25 [-0.38, -0.12] (8:0; $p = 0.008$)	0.14 [0.02, 0.28] (0:5; $p = 0.062$)	-0.25 [-0.38, -0.12] (8:0; $p = 0.008$)
fault recall	-0.31 [-0.35, -0.27] (12:0; $p < 0.001$)	-0.68 [-0.82, -0.53] (12:0; $p < 0.001$)	-0.28 [-0.44, -0.14] (9:0; $p = 0.004$)	0.00 [0.00, 0.00] (0:0; $p = 1.000$)
crit. FA/h	3,065 [2,007, 4,008] (11:0; $p < 0.001$)	93.4 [60.8, 123.2] (10:0; $p = 0.002$)	122.1 [81.0, 162.8] (10:0; $p = 0.002$)	0.4 [-1.6, 2.4] (2:2; $p = 1.000$)
degraded FA/h	1,754 [863.4, 2,754] (9:3; $p = 0.004$)	-123.5 [-127.3, -119.7] (0:12; $p < 0.001$)	-108.8 [-123.3, -87.4] (0:12; $p < 0.001$)	-97.7 [-101.2, -93.7] (0:12; $p < 0.001$)
states FA/h	4,829 [3,863, 5,972] (12:0; $p < 0.001$)	104.1 [90.3, 119.0] (12:0; $p < 0.001$)	119.7 [87.4, 153.8] (11:0; $p < 0.001$)	118.3 [113.9, 121.8] (12:0; $p < 0.001$)

F. Short defects

Persistence sets a floor on the length of a defect a policy can catch. In runs with 20 visual defects of one fixed length (same units, nominal sensors), the full policy found 0.10 of 1-frame, 0.43 of 2-frame and 0.70 of 3-frame defects and reached a mean recall of 0.95 from 8 frames (267 ms) on. The delay-timer, which suppressed glare best, needed 15 frames (500 ms) and found only 0.07 of 3-frame defects; single-frame thresholding found 0.96 of 1-frame defects. Glare suppression and short-defect recall are thus two sides of the same trade-off [8]; a part that stays in view for fewer frames than these lengths can pass unflagged.

G. Scenario analysis: operator load

The workloads compress many events into five minutes, so their hourly alert rates overstate a real line. Fig. 2 is therefore a scenario analysis, not a measurement: it combines the per-event measurements with two *assumed* rates: 30 glare bursts per hour and $\mu = 60$ reviews per hour. Under an $M/M/1$ model [36], one reviewer is overloaded by the baseline even without defects and by EMA_KOFN because of glare alone. The full policy stays below $\rho = 1$ up to about 40 defect episodes per hour, the delay-timer up to about 44 and decision fusion up to about 55, because the two baselines raise almost no glare alerts; on this metric the simple baselines are better. This is a model of utilization, not an observation of operators; the queue model assumes Poisson arrivals, whereas alerts are burstier, and operators also discount unreliable alarms [37], [38].

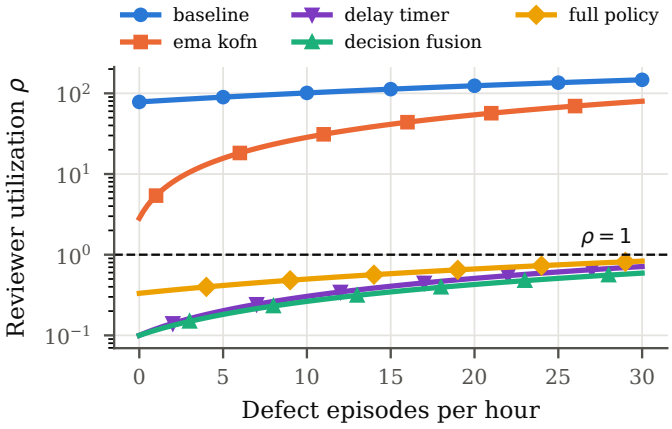


Fig. 2. Scenario analysis (not an observation of operators): modeled utilization of one reviewer (assumed $\mu = 60$ reviews/h) as a function of the defect-episode rate, using the measured alerts per episode, alerts per glare burst (assuming 30 bursts/h) and nominal false-alert rate of each policy (independent frame scores). Values above $\rho = 1$ mean an unbounded backlog under an $M/M/1$ model.

H. Sensitivity

Fig. 3 varies one parameter at a time. With the revised rule (4), critical false escalations stayed between 3.4 and 4.0

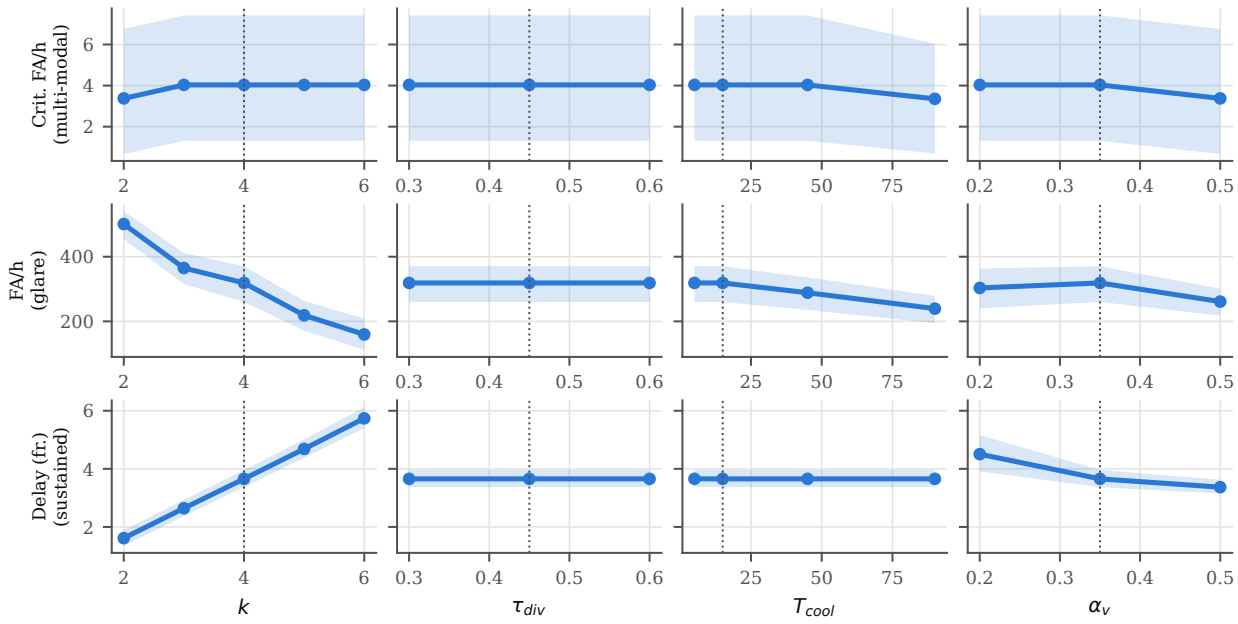


Fig. 3. One-at-a-time sensitivity of the full policy (mean and 95% bootstrap band over 21 units; dotted line: fixed default). Rows: false critical alerts per hour (multi-modal workload), false alerts per hour (glare workload), mean detection delay (sustained defects).

per hour for every tested value of every parameter, so they no longer depend on τ_{div} or α_v. The parameters mainly trade glare alerts against delay: raising k from 2 to 6 lowered false alerts in the glare workload from 501 to 159 per hour while the mean delay grew from 1.6 to 5.7 frames, and longer incidents (T_{cool} up to 90 frames) lowered them from 319 to 239 per hour. Recall on the multi-modal workload never fell below 1.00. The bands in this figure resample units, not categories, and are therefore narrower than the category-level intervals above.

I. Latency

Table VII times every stage with a real PatchCore forward pass on an RTX 4050 Laptop GPU. With synchronous persistence the mean cycle took 27.3 ms (p99 44.4 ms) and 2.46% of cycles exceeded the 33.3 ms frame budget; single spool and audit inserts occasionally took up to 30 and 31 ms. With the writer thread, enqueuing took at most 0.20 ms, the mean cycle took 26.8 ms (p99 33.3 ms), and 1.00% of cycles missed the budget. The worst cycle did not improve (56.8 vs. 61.2 ms): the remaining misses come from PNG decoding (up to 32.2 ms) and the vision stage (up to 31.1 ms). The price is durability: a decision reached disk at most 66.8 ms after it was enqueued (p99 16.7 ms; the writer queue never held more than 1 pending decision), and would be lost in a crash within that window. Neither configuration is hard real-time. On the CPU alone the cycle took 59.2 ms with the writer thread and missed the budget in 100% of cycles. On embedded hardware, the companion study [39] compiled the same ResNet-18 PatchCore scoring path to TensorRT 10.16 in FP16 on a Jetson Orin Nano (15 W mode): paced at 30 FPS, the scoring pipeline took a median of 8.2 ms per frame (p95 8.6 ms) including host–device copies, reached 243 FPS unpaced, and the module drew 3.98 W (0.85 W above idle), with decisions identical to FP32 on its test frames.

J. Durable delivery

Table VIII reports six fault cases. Link losses of up to 120 s (3,751 events), a broker restart and a SIGKILL of the publisher during draining lost no event, delivered none out of order, and drained within 2.0 s after recovery. When a 60 s outage exceeded a 1,000-record spool, the spool evicted 801 events and 801 were missing, so every loss was accounted for (0 unexplained). Because the spool counts but does not record evicted identifiers, an evicted event that had already been sent could still arrive; this happened 0 times in this run. The crash run produced 0 duplicates; duplicates are possible by design and are removed by identifier. These cases cover the publisher and the broker.

Table IX extends the tests to the subscriber and the inspection process. With a random client identifier and a clean session, which is how our subscriber connected before this revision, a 10 s subscriber crash lost 308 events: all 300 sent while it was down and a few sent while it reconnected, although every one of them had left the spool with a broker acknowledgment. With a persistent session, a 10 s crash lost 0 events. A 60 s crash, during which 1,800 events were sent, lost 807 events under Mosquitto's default limit of 1,000 queued messages per client, and 0 with the limit removed. The broker's queue limit therefore bounds how long a subscriber may be down (about 33 s at 30 events/s with the default), and the edge spool cannot recover what the broker drops, because those events were already acknowledged; our broker configuration now raises the limit to one hour of events. Killing the inspection process 100 times lost 0 of 6,604 decisions with synchronous persistence

TABLE VII

PER-CYCLE LATENCY ON THE INSPECTION THREAD, PACED AT 30 FRAMES/S (5,000 GPU AND 500 CPU CYCLES PER CONFIGURATION; 700 × 700 PNG FRAMES; MEMORY BANK 13,798 PATCHES). SYNCHRONOUS: SPOOL AND AUDIT INSERTS ON THE INSPECTION THREAD. WRITER THREAD: THE INSPECTION THREAD ONLY ENQUEUES THE DECISION.

	GPU, synchronous			GPU, writer thread			CPU, writer thread	
Stage (ms)	mean	p99	max	mean	p99	max	mean	p99
PNG decode	9.67	13.29	32.2	9.60	13.03	31.2	10.35	13.75
Optical check + PatchCore	16.88	21.00	31.1	16.95	20.02	32.8	48.57	89.85
Sensor step	0.11	0.23	1.4	0.11	0.23	1.9	0.13	0.29
Policy	0.06	0.12	0.5	0.06	0.12	0.4	0.06	0.15
Spool insert	0.33	0.62	29.6	–	–	–	–	–
Audit insert	0.27	5.11	30.6	–	–	–	–	–
Enqueue to writer	–	–	–	0.05	0.08	0.2	0.04	0.07
End to end	27.32	44.43	56.8	26.77	33.31	61.2	59.15	101.42
Deadline misses			2.46%			1.00%		100%

TABLE VIII

DELIVERY THROUGH A REAL MOSQUITTO 2.0.18 BROKER AT 30 EVENTS/S (QOS 1, SYNCHRONOUS SPOOL INSERTS). MISS.: EVENTS NEVER RECEIVED; EVICT.: REMOVED BY THE SPOOL CAPACITY; DUP.: REPEATED DELIVERIES; ORDER: FIRST DELIVERIES OUT OF SEQUENCE; PEAK: LARGEST SPOOL DEPTH.

Fault	Events	Miss.	Evict.	Dup.	Order	Peak	Drain (s)
Link loss 30 s	1,051	0	0	0	0	901	0.71
Link loss 60 s	1,951	0	0	0	0	1,801	1.06
Link loss 120 s	3,751	0	0	0	0	3,601	1.97
Broker restart 30 s	1,051	0	0	0	0	901	1.66
Publisher SIGKILL	600	0	0	0	0	600	1.61
Overflow (cap. 1,000)	1,951	801	801	0	0	1,000	0.57

TABLE IX

FAILURES BEYOND THE PUBLISHER AND BROKER (SAME BROKER, 30 EVENTS/S, QOS 1). TOP: THE AUDIT-LOG SUBSCRIBER IS KILLED WITH SIGKILL AFTER 5 S AND RESTARTED AFTER THE STATED TIME; QUEUE: THE BROKER'S LIMIT ON QUEUED MESSAGES PER OFFLINE CLIENT (1,000 IS THE MOSQUITTO DEFAULT). MISSING: EVENTS NEVER STORED IN THE AUDIT LOG. BOTTOM: THE INSPECTION PROCESS IS KILLED AT A RANDOM TIME 1–3 S AFTER START; HANDED OFF: DECISIONS WHOSE SPOOL INSERT (SYNCHRONOUS) OR QUEUE SUBMISSION (WRITER THREAD) HAD RETURNED; LOST: HANDED OFF BUT NOT IN THE SPOOL AFTERWARDS (KILLS WITH A LOSS).

Subscriber SIGKILL	Down (s)	Sent while down	Missing
Clean session	10	300	308
Persistent, queue 1,000	10	300	0
Persistent, queue 1,000	60	1,800	807
Persistent, unlimited	60	1,800	0
Inspection SIGKILL	**Kills**	**Handed off**	**Lost (kills)**
Synchronous	100	6,604	0 (0)
Writer thread	100	5,801	0 (0)

and 0 of 5,801 with the writer thread. The writer's exposure window is short compared with the 33 ms frame period, so a loss is rare rather than impossible: zero losses in 100 kills only bound the probability of a loss per crash below about 3% (95% confidence). Power loss was not tested. It is the case `synchronous=NORMAL` does not cover; on our disk, `synchronous=FULL`, which syncs every commit, raised the median spool insert from 0.02 to 4.19 ms (p99 0.14 vs. 15.27 ms, 2,000 inserts), which only fits the frame budget if the writes run on the writer thread. We did not test exactly-once processing, which the design does not provide.

TABLE X

REPLAY OF TWO SYNTHETIC RUN-TO-FAILURE TRACES (600 FRAMES, SHAPE INSPIRED BY THE IMS BEARING [40] AND C-MAPSS [41] DATA BUT NOT DERIVED FROM THEM) WITH GOOD-PART VISUAL SCORES, MEANS OVER THE SEVEN CATEGORIES. FIRST FLAG: FIRST NON-NORMAL DECISION RELATIVE TO THE LABELED ONSET (NEGATIVE = BEFORE). ALERTS BEFORE/FROM THE ONSET.

		first flag	alerts		HIGH alerts
Trace	Policy	(fr. vs. onset)	before	from onset	from onset
Bearing proxy	`BASELINE`	-337	14.1	7.7	0.0
	`DELAY_TIMER`	never	0.0	0.0	0.0
	`DECISION_FUSION`	-99	1.0	0.0	0.0
	`NO_FUSION`	never	0.0	0.0	0.0
	`FULL_POLICY`	-90	1.0	0.0	0.0
Thermal-creep proxy	`BASELINE`	-307	12.9	9.0	0.0
	`DELAY_TIMER`	never	0.0	0.0	0.0
	`DECISION_FUSION`	-94	1.0	0.0	0.0
	`NO_FUSION`	never	0.0	0.0	0.0
	`FULL_POLICY`	-92	1.0	0.0	0.0

K. Synthetic sensor traces

Table X checks the sensor path in isolation. The full policy first flagged both traces 90–92 frames *before* the labeled onset, during the incipient drift, with a single REVIEW alert; it raised no further alert and no HIGH alert after the onset, because rule (5) routes sensor-only evidence to review. Whether that early review counts as a timely detection or a false alert depends on where the fault is said to begin. A vision-only policy cannot see these faults; the baseline's alerts are chance crossings of nominal scores.

VI. THREATS TO VALIDITY

Data. The main results use 7 curated MVTec AD categories with small good-image pools; the held-out VisA rerun (Section V-E) adds 12 harder categories, but Real-IAD [22] and real production data remain untested. With the same ResNet-18 PatchCore at the 99th-percentile threshold, the companion study [39] measured a median per-image recall of 0.94 on MVTec AD but 0.57 on the 12 VisA [21] categories and 0.72

on VisA at 448×448 input. The policy can delay or merge detections but not create them, so such detector losses carry over to the alert stream. *Temporal realism.* Scores are sampled from pools rather than taken from video; the correlated rerun shows that conclusions about false alerts on good parts depend strongly on this choice. Glare is synthetic, sensors are simulated and the traces are synthetic. *Workload density.* Event densities are much higher than on a real line, which is why Section V-G models load per event; the load model uses assumed rates and is not an observation of operators. *Optical check.* The blur check misfired on real defects: in metal_nut it flagged 19.4% of defective test images (flip) as blurred, which would route those parts to a camera-maintenance review, while no good test image was flagged; the workloads run the check on good training frames and therefore do not exercise this failure. *Statistics.* Seven MVTec categories are few clusters; category-level intervals are wide, per-category differences in Table II are often ties, and the permutation tests cannot go below $p = 0.016$. We report p-values without multiplicity correction and read them as descriptive. *Post-hoc change.* Rule (4) was revised after inspecting the first MVTec ablation; the external baselines were defined before they were run, and the VisA evaluation was run only after everything was frozen. *Systems.* Latency was measured on one laptop GPU without camera acquisition or network transport. Delivery is at-least-once, not exactly-once; power loss was not tested, and the crash tests killed processes on one machine, not a networked deployment. The subscriber's persistent session was added in this revision after the crash test exposed the loss; both configurations were measured in the same benchmark run.

VII. Conclusion

Wrapping a visual anomaly detector in alarm-management primitives is useful only in combination, and no single design dominated. The integrated policy gave the fewest false line-stop escalations, used sensors to see faults the camera cannot, and handled machine states and degraded inputs that simple alarms ignore; a plain delay-timer suppressed short glare far better at the cost of missing defects shorter than about half a second; every persistence-based design lost its zero false-alert result once consecutive scores were correlated; and on held-out VisA images the machine-state and glare findings replicated, but the line-stop advantage over decision fusion did not and every persistence-based policy missed many sustained defects within 0.5 s. Acknowledgment-driven spooling kept every event through the publisher-side outages and crashes we tested, but end-to-end durability also depends on the subscriber's session and the broker's queue limit, which lost events when the subscriber was down; a writer thread more than halved the frame-budget misses at the price of a small crash window, without removing the worst-case cycle. The most important open questions are how these rankings change on real, temporally correlated inspection video, and how thresholds should be set when defect scores are as weak as on VisA. Code, configuration, scores and every generated table and figure are available at https://github.com/sengarutk/edge-quality-intelligence; `scripts/reproduce_all_paper_results.sh` regenerates them.

References

[1] K. Roth, L. Pemula, J. Zepeda, B. Schölkopf, T. Brox, and P. Gehler, "Towards total recall in industrial anomaly detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 14 298–14 308.

[2] T. Defard, A. Setkov, A. Loesch, and R. Audigier, "PaDiM: A patch distribution modeling framework for anomaly detection and localization," in *Pattern Recognition. ICPR International Workshops and Challenges*, ser. Lecture Notes in Computer Science. Springer, 2021, pp. 475–489.

[3] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, "MVTec AD – a comprehensive real-world dataset for unsupervised anomaly detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 9584–9592.

[4] P. Bergmann, K. Batzner, M. Fauser, D. Sattlegger, and C. Steger, "The MVTec anomaly detection dataset: A comprehensive real-world dataset for unsupervised anomaly detection," *International Journal of Computer Vision*, vol. 129, no. 4, pp. 1038–1059, 2021.

[5] J. Chen and X. Ran, "Deep learning with edge computing: A review," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1655–1674, 2019.

[6] *ANSI/ISA-18.2-2016: Management of Alarm Systems for the Process Industries*, International Society of Automation Std., 2016.

[7] Engineering Equipment and Materials Users Association, *EEMUA Publication 191: Alarm Systems – A Guide to Design, Management and Procurement*, 3rd ed. London: EEMUA, 2013.

[8] N. A. Adnan, I. Izadi, and T. Chen, "On expected detection delays for alarm systems with deadbands and delay-timers," *Journal of Process Control*, vol. 21, no. 9, pp. 1318–1331, 2011.

[9] N. A. Adnan, Y. Cheng, I. Izadi, and T. Chen, "Study of generalized delay-timers in alarm configuration," *Journal of Process Control*, vol. 23, no. 3, pp. 382–395, 2013.

[10] M. S. Afzal, T. Chen, A. Bandehkhoda, and I. Izadi, "Analysis and design of time-deadbands for univariate alarm systems," *Control Engineering Practice*, vol. 71, pp. 96–107, 2018.

[11] J. Wang, F. Yang, T. Chen, and S. L. Shah, "An overview of industrial alarm systems: Main causes for alarm overloading, research status, and open problems," *IEEE Transactions on Automation Science and Engineering*, vol. 13, no. 2, pp. 1045–1061, 2016.

[12] P. Goel, A. Datta, and M. S. Mannan, "Industrial alarm systems: Challenges and opportunities," *Journal of Loss Prevention in the Process Industries*, vol. 50, pp. 23–36, 2017.

[13] J. Wang, Z. Wang, X. Zhou, and F. Yang, "Design of delay timers based on estimated probability mass functions of alarm durations," *Journal of Process Control*, vol. 110, pp. 154–165, 2022.

[14] J. Su, C. Guo, H. Zang, F. Yang, D. Huang, X. Gao, and Y. Zhao, "A multi-setpoint delay-timer alarming strategy for industrial alarm monitoring," *Journal of Loss Prevention in the Process Industries*, vol. 54, pp. 1–9, 2018.

[15] P. Gyasi and J. Wang, "Design of serial alarm systems based on deadbands and delay timers for removing false alarms," *Process Safety and Environmental Protection*, vol. 162, pp. 1033–1041, 2022.

[16] Z. Zhang, J. Wang, and Y. Qi, "Selection of alarm deadbands and delay timers with their connections based on risk indicators for removing nuisance alarms," *Control Engineering Practice*, vol. 153, p. 106113, 2024.

[17] J. Zhou, J. Shang, and T. Chen, "Performance analysis for stochastic anomaly detectors with on/off-delay timers," *Automatica*, vol. 173, p. 112073, 2025.

[18] V. Heydari and K. Nyarko, "Alarm-budgeted event-level evaluation of ICS anomaly detection: Lessons from SWaT and WADI," *IEEE Access*, vol. 14, pp. 17 585–17 609, 2026.

[19] R. Yu, M. Wang, J. Yun, J. Du, L. Wang, and Y. Suo, "Unified reproduction and event-level evaluation of industrial multivariate time-series anomaly detection methods," in *7th International Conference on Computing, Networks and Internet of Things (CNIOT)*, 2026, pp. 1–5.

[20] E. S. Sadik and H. T. Canseven, "Onset-aware alarm-triggered fault diagnosis in collaborative robots," *IEEE Access*, vol. 14, pp. 115 596–115 608, 2026.

[21] Y. Zou, J. Jeong, L. Pemula, D. Zhang, and O. Dabeer, "SPot-the-Difference self-supervised pre-training for anomaly detection and segmentation," in *Computer Vision – ECCV 2022*, ser. Lecture Notes in Computer Science. Springer, 2022, pp. 392–408.

[22] C. Wang, W. Zhu, B.-B. Gao, Z. Gan, J. Zhang, Z. Gu, S. Qian, M. Chen, and L. Ma, "Real-IAD: A real-world multi-view dataset for benchmarking versatile industrial anomaly detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 22 883–22 892.

[23] Z. Li, Y. Yan, X. Wang, Y. Ge, and L. Meng, "A survey of deep learning for industrial visual anomaly detection," *Artificial Intelligence Review*, vol. 58, 2025.

[24] A. Cools, M. A. Belarbi, and S. A. Mahmoudi, "Benchmarking of anomaly detection methods for Industry 4.0: Evaluation, ranking, and practical recommendations," *Big Data and Cognitive Computing*, vol. 9, p. 128, 2025.

[25] A. Stropeni, F. Genilotti, F. Borsatti, M. Barusco, D. D. Pezze, and G. A. Susto, "Efficient visual anomaly detection at the edge: Enabling real-time industrial inspection on resource-constrained devices," in *12th International Conference on Control, Decision and Information Technologies (CoDIT)*, 2026, pp. 849–854.

[26] Y. Lin, W. Lin, and Y. Lin, "Real-time multimodal defect detection and MES feedback on edge devices via Bayesian fusion and causal adaptation," *IEEE Access*, vol. 13, pp. 162 226–162 237, 2025.

[27] *MQTT Version 3.1.1*, OASIS Std., 2014, oASIS Standard.

[28] H. Koziolek, S. Grüner, and J. Rückert, "A comparison of MQTT brokers for distributed IoT edge computing," in *Software Architecture (ECSA 2020)*, ser. Lecture Notes in Computer Science. Springer, 2020, pp. 352–368.

[29] S. Grüner, H. Koziolek, and J. Rückert, "Towards resilient IoT messaging: An experience report analyzing MQTT brokers," in *IEEE 18th International Conference on Software Architecture (ICSA)*, 2021, pp. 69–79.

[30] A. Mohammed, R. S. S. Singh, S. Aslam, and Y. C. Wong, "A scalable MQTT-based edge IoT architecture for real-time distributed solar PV panel monitoring," *Engineering, Technology & Applied Science Research*, vol. 16, pp. 36 014–36 024, 2026.

[31] J. A. Matias and R. J. d. A. Macêdo, "Resilient IIoT communication via edge-based store-and-forward and SLO monitoring," in *Anais do XXVII Workshop de Testes e Tolerância a Falhas (WTF)*, 2026, pp. 242–246.

[32] T. F. Gonzalez, "Clustering to minimize the maximum intercluster distance," *Theoretical Computer Science*, vol. 38, pp. 293–306, 1985.

[33] J. L. Pech-Pacheco, G. Cristóbal, J. Chamorro-Martínez, and J. Fernández-Valdivia, "Diatom autofocusing in brightfield microscopy: A comparative study," in *Proceedings of the 15th International Conference on Pattern Recognition (ICPR)*, vol. 3, 2000, pp. 314–317.

[34] SQLite, "Write-ahead logging," https://www.sqlite.org/wal.html, accessed 2026.

[35] C. A. Field and A. H. Welsh, "Bootstrapping clustered data," *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, vol. 69, no. 3, pp. 369–390, 2007.

[36] J. F. Shortle, J. M. Thompson, D. Gross, and C. M. Harris, *Fundamentals of Queueing Theory*, 5th ed. Hoboken, NJ: John Wiley & Sons, 2018.

[37] J. P. Bliss, R. D. Gilson, and J. E. Deaton, "Human probability matching behaviour in response to alarms of varying reliability," *Ergonomics*, vol. 38, no. 11, pp. 2300–2312, 1995.

[38] C. D. Wickens, S. Rice, D. Keller, S. Hutchins, J. Hughes, and K. Clayton, "False alerts in air traffic control conflict alerting system: Is there a "cry wolf" effect?" *Human Factors*, vol. 51, no. 4, pp. 446–462, 2009.

[39] U. Sengar, "When does edge AI defect inspection pay back its carbon? Measured embedded inference, detector recall, and alert design," 2026, manuscript in preparation. Code and results: https://github.com/sengarutk/sustainable-edge-quality-intelligence.

[40] H. Qiu, J. Lee, J. Lin, and G. Yu, "Wavelet filter-based weak signature detection method and its application on rolling element bearing prognostics," *Journal of Sound and Vibration*, vol. 289, no. 4–5, pp. 1066–1090, 2006.

[41] A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," in *International Conference on Prognostics and Health Management (PHM)*, 2008, pp. 1–9.